

Code Break-Down: **HC-EntityEngine.xml**

Chunk 1: Root Element — Starting Point

```
<entity-config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation="https://ofbiz.apache.org/dtds/entity-config.xsd">
```

What it does

This is the root tag of the configuration file.

Internally OFBiz reads this XML when it starts (`./gradlew ofbiz`) and parses it using its XML parser.

Breakdown:

`xmlns:xsi`

Defines XML schema namespace.

Namespace prevents naming conflicts.

Think:

`<table>`

may exist in multiple systems.

Namespace tells XML: "Which table definition are we referring to?"

`xsi:noNamespaceSchemaLocation`

Points to:

`https://ofbiz.apache.org/dtds/entity-config.xsd`

This `.xsd` file defines:

- what tags are allowed
- their order
- mandatory attributes
- validation rules

Internally:

When OFBiz loads:

1. XML parser reads entity-config.xml
2. XML validator checks against `.xsd`
3. If syntax is wrong:

Example:

`<datasource>`

instead of:

`<datasource>`

OFBiz throws a startup error.

So internally:

Read XML



Validate with XSD



Create Java objects from XML

Chunk 2: Resource Loader

Code:

```
<resource-loader  
    name="fieldfile"  
    class="org.apache.ofbiz.base.config.FileLoader"  
    prepend-env="ofbiz.home"  
    prefix="/framework/entity/fieldtype/" />
```

What it does

Defines a loader responsible for loading files.

Internally OFBiz creates:

```
FileLoader loader = new FileLoader();
```

with:

```
name="fieldfile"
```

Later OFBiz can call:

```
fieldfile.load(...)
```

instead of specifying a path every time.

Attributes

name

```
name="fieldfile"
```

Identifier.

Later references use:

loader="fieldfile"

class

class="org.apache.ofbiz.base.config.FileLoader"

Actual Java class to instantiate.

Internally:

Reflection is used:

Class.forName(

"org.apache.ofbiz.base.config.FileLoader"

).newInstance();

OFBiz dynamically creates object.

prepend-env

prepend-env="ofbiz.home"

Means:

Take OFBiz home directory first.

Suppose:

/home/user/ofbiz

prefix

prefix="/framework/entity/fieldtype/"

Final path becomes:

/home/user/ofbiz/framework/entity/fieldtype/

When later:

location="fieldtypemysql.xml"

OFBiz internally builds:

/home/user/ofbiz/framework/entity/fieldtype/fieldtypemysql.xml

Chunk 3: Transaction Factory

Code:

```
<transaction-factory
```

```
class="org.apache.ofbiz.entity.transaction.GeronimoTransactionFactory"/>
```

Purpose

Controls database transactions.

Transaction example:

Suppose service does:

insertOrder()

updateInventory()

sendInvoice()

If invoice fails:

Without transaction:

Order inserted

Inventory updated

Invoice failed

Database becomes inconsistent.

With transaction:

Start transaction



Insert order



Update inventory



Send invoice



Failure



Rollback

Everything returns back.

Internally:

OFBiz creates:

GeronimoTransactionFactory factory

Then:

UserTransaction tx=

factory.getUserTransaction();

Later services use:

tx.begin();

```
tx.commit();
```

```
tx.rollback();
```

Chunk 4: Connection Factory

Code:

```
<connection-factory
```

```
class="org.apache.ofbiz.entity.connection.DBCPConnectionFactory"/>
```

Purpose

Responsible for creating database connections.

Without pooling:

Every query:

```
Connection c=
```

```
DriverManager.getConnection();
```

Problem:

Connection creation is expensive.

OFBiz uses:

DBCPConnectionFactory

DBCP:

Database Connection Pool

Instead of:

Create

Destroy

Create

Destroy

it keeps ready connections:

Pool:

Connection1

Connection2

Connection3

Connection4

Request:

connection=pool.getConnection()

After use:

pool.returnConnection()

Not destroyed.

Performance improves.

Chunk 5: XA Debug

Code:

```
<debug-xa-resources value="false"/>
```


XA means:

Extended Architecture transactions

Used for distributed transactions.

Example:

Database1

Database2

Message queue

All should succeed together.

Internally:

If true:

Debug.log()

prints detailed transaction information.

Currently:

false

so logging disabled.

Chunk 6: Delegator (very important)

Code:

```
<delegator
```

```
name="default"
```

```
entity-model-reader="main"
```

entity-group-reader="main"

entity-eca-reader="main"

distributed-cache-clear-enabled="false">

<group-map

group-name="org.apache.ofbiz"

datasource-name="localmysql"/>

</delegator>

This is one of OFBiz's core concepts.

Delegator acts as:

Middleman between

Application ↔ Database

Application never directly writes SQL.

Instead:

delegator.findOne()

delegator.create()

delegator.store()

Internally:

When:

delegator.findOne("Product")

runs:

Step 1:

Delegator checks entity model:

Product entity belongs to group:

org.apache.ofbiz

↓

Step 2:

Group mapping:

datasource-name="localmysql"

↓

Step 3:

Uses datasource:

localmysql

↓

Step 4:

Creates SQL:

SELECT * FROM PRODUCT

↓

Step 5:

Executes query.

Flow:

Delegator.findOne()

↓

Entity model



Group mapping



Datasource



Database

The `default-no-eca` delegator works similarly but disables Entity ECA rules:

`entity-eca-enabled="false"`

Normally:

Create order



Trigger notifications



Trigger workflows

With ECA disabled:

Create order only

Used for bulk imports.

Continuing from where we stopped. Now we'll move into **Entity Readers, Data Readers, Datasource configuration, and internal database flow**. This is where OFBiz decides *what data to load and how to connect to the database*.

Chunk 7: Entity Model Reader

Code:

```
<entity-model-reader name="main"/>
```

What it does

This tells OFBiz:

"Use a model reader called `main`."

At first glance it looks small, but internally it becomes very important.

Entity model reader loads all entity definitions from files like:

entitymodel.xml

Example:

```
<entity entity-name="Product">
  <field name="productId" type="id"/>
  <field name="productName" type="name"/>
</entity>
```

Internal working

When OFBiz starts:

```
EntityModelReader reader =
```

```
EntityModelReader.getReader("main");
```

Then:

```
reader.loadEntities();
```

Internally:

Search components

↓

Find entitymodel.xml files



Parse XML



Create ModelEntity objects



Store in memory

Memory now contains:

Product

{

productId,

productName

}

Now when:

delegator.findOne("Product")

runs,

OFBiz already knows:

- table name
- fields
- primary keys
- relationships

without querying database.

Think of it like:

Entity Model Reader

XML definitions



Java objects in memory



Used by Delegator

Chunk 8: Entity Group Reader

Code:

```
<entity-group-reader name="main"/>
```

Purpose

Groups entities together.

Imagine system has:

Product

Order

Customer

Analytics

Logs

You may not want all tables in one database.

Maybe:

Database A:

Products

Orders

Database B:

Analytics

Logs

Group reader helps organize this.

Internally:

OFBiz loads:

entitygroup.xml

Example:

```
<entity-group  
  entity="Product"  
  group="org.apache.ofbiz"/>
```

Then creates mapping:

Map<String,String>

Like:

Product → org.apache.ofbiz

Order → org.apache.ofbiz

Later Delegator checks:

Product

↓

org.apache.ofbiz



localmysql



Database connection

Flow:

Entity



Group Reader



Group Name



Datasource

Chunk 9: Entity ECA Reader

Code:

```
<entity-eca-reader name="main"/>
```

ECA means:

Event Condition Action

Very similar to triggers in databases.

Example:

Customer created

Automatically:

Send email

Create audit log

Create welcome notification

Internally OFBiz loads:

entityeca.xml

Suppose:

```
<eca entity="OrderHeader"
  operation="create">
```

```
<action service="sendOrderEmail"/>
```

```
</eca>
```

Internally:

OFBiz stores:

OrderHeader.create()

↓

sendOrderEmail()

Now when:

delegator.create("OrderHeader")

runs:

Execution becomes:

Create record



Check ECA rules



Rule found



Call service

Without ECA:

Create order

With ECA:

Create order



Send mail



Audit log



Notification

Chunk 10: Entity Data Readers

Code:

```
<entity-data-reader name="tenant"/>
<entity-data-reader name="seed"/>
<entity-data-reader name="seed-initial"/>
<entity-data-reader name="demo"/>
<entity-data-reader name="ext"/>
<entity-data-reader name="ext-test"/>
<entity-data-reader name="ext-demo"/>
```

These define categories of data OFBiz can load.

Internally OFBiz creates:

EntityDataReader

objects:

tenant

seed

demo

ext

Seed data

Contains:

Countries

Currency

Status types

Roles

These are necessary.

When:

```
./gradlew loadAll
```

runs:

Internally:

```
reader.loadData("seed");
```

OFBiz searches:

```
data/seed/
```

Reads XML:

```
<Product productId="P100"/>
```

Creates SQL:

```
INSERT INTO PRODUCT
```

```
VALUES(...)
```

Demo data

Contains sample records:

Demo users

Demo products

Demo orders

Useful for learning.

Tenant

For multi-tenant systems.

Example:

Company A

Company B

Each may have separate data.

Flow:

loadAll



Read XML files



Convert to GenericValue



Generate SQL



Insert into DB

Chunk 11: Field Type Definitions

Code:

```
<field-type
```

```
name="derby"
```

```
loader="fieldfile"
```

```
location="fieldtypederby.xml"/>
```

```
<field-type
```

```
name="mysql"
```

```
loader="fieldfile"
```

```
location="fieldtypemysql.xml"/>
```

Purpose

Different databases use different datatype names.

Example:

Java:

String

MySQL:

VARCHAR(255)

Derby:

LONG VARCHAR

OFBiz needs translation.

Internally:

When:

```
<field name="productName"
```

```
type="name"/>
```

OFBiz checks:

Current DB:

mysql

Then loads:

fieldtypemysql.xml

Finds:

```
<field-type-def
```

```
  type="name"
```

```
  sql-type="VARCHAR(255)"/>
```

Creates:

```
productName VARCHAR(255)
```

Flow:

Entity field



Field type



Database specific type



SQL generation

Chunk 12: Internal Flow Until Now

At startup OFBiz roughly does:

Start OFBiz



Read entity-config.xml



Create transaction factory



Create connection factory



Load entity models



Load group mappings



Load ECA rules



Load field types



Create delegator



Ready for database operations

Chunk 13: Datasource Definition

Code:

```
<datasource name="localmysql"
```

```
helper-class="org.apache.ofbiz.entity.datasource.GenericHelperDAO"
field-type-name="mysql"
check-on-start="true"
add-missing-on-start="true"
check-pks-on-start="false"
use-foreign-keys="true"
join-style="ansi-no-parenthesis"
alias-view-columns="false"
drop-fk-use-foreign-key-keyword="true"
table-type="InnoDB"
character-set="utf8mb4"
collate="utf8mb4_0900_ai_ci">
```

This is basically telling OFBiz:

"For this datasource, use MySQL and these rules."

Internally OFBiz creates:

GenericHelperInfo helperInfo

which stores:

Datasource name = localmysql

Database type = mysql

Charset = utf8mb4

Table type = InnoDB

OFBiz keeps this object in memory.

Chunk 14: Helper Class

Code:

```
helper-class="org.apache.ofbiz.entity.datasource.GenericHelperDAO"
```

This class is extremely important.

Internally:

OFBiz creates:

```
GenericHelperDAO helper =  
new GenericHelperDAO();
```

Think of it as SQL generator + executor.

Whenever:

```
delegator.create()
```

```
delegator.findOne()
```

```
delegator.store()
```

is called:

Delegator doesn't generate SQL itself.

Instead:

delegator

↓

GenericHelperDAO

↓

SQL

↓

Database

Suppose code says:

```
delegator.findOne(  
    "Product",  
    UtilMisc.toMap(  
        "productId",  
        "P100"  
    ))
```

Internally:

GenericHelperDAO creates:

```
SELECT *  
  
FROM PRODUCT  
  
WHERE PRODUCT_ID='P100'
```

Then:

```
PreparedStatement ps=  
connection.prepareStatement(sql);
```

Then:

```
ResultSet rs=  
  
ps.executeQuery();
```

Then converts results:

```
GenericValue product=  
  
new GenericValue();
```

Flow:

Delegator



GenericHelperDAO



PreparedStatement



SQL Execution



ResultSet



GenericValue

Chunk 15: Startup Database Checks

Code:

check-on-start="true"

add-missing-on-start="true"

check-pks-on-start="false"

These control what OFBiz checks during startup.

check-on-start

check-on-start="true"

When OFBiz starts:

Internally:

```
helper.checkDataSource();
```

OFBiz asks database:

```
SHOW TABLES
```

Then compares with entity definitions.

Example:

Entity:

```
<entity entity-name="Product">
```

Expected table:

```
PRODUCT
```

If missing:

Database error detected

add-missing-on-start

```
add-missing-on-start="true"
```

Suppose XML says:

```
<field
```

```
name="description"
```

```
type="description"/>
```

but database doesn't contain:

```
DESCRIPTION
```

Internally:

OFBiz detects:

Field missing

Creates SQL:

```
ALTER TABLE PRODUCT
```

```
ADD DESCRIPTION VARCHAR(255)
```

Automatically fixes schema.

Flow:

Startup



Compare XML vs DB



Missing field found



Generate ALTER TABLE

check-pks-on-start

check-pks-on-start="false"

Primary key checking disabled.

Why?

Checking PKs for hundreds of tables increases startup time.

If enabled:

checkPrimaryKeys()

would scan all entities.

Chunk 16: Foreign Keys

Code:

```
use-foreign-keys="true"
```

Suppose:

Customer table:

Customer

customerId

name

Order table:

Order

orderId

customerId

customerId refers to Customer.

Internally OFBiz creates:

```
ALTER TABLE ORDER
```

```
ADD CONSTRAINT FK_ORDER_CUSTOMER
```

```
FOREIGN KEY(customerId)
```

```
REFERENCES CUSTOMER(customerId)
```

Why important?

Without FK:

Order

customerId=XYZ999

even if customer doesn't exist.

Database allows invalid data.

With FK:

Insert Order

↓

Customer exists?

↓

No

↓

Reject

Chunk 17: Join Style

Code:

```
join-style="ansi-no-parenthesis"
```

Controls SQL join syntax generation.

Suppose:

```
delegator.findList(
```

"OrderAndCustomer"

)

Internally:

OFBiz generates SQL.

Possible styles:

Old:

FROM ORDER, CUSTOMER

WHERE ORDER.customerId=

CUSTOMER.customerId

ANSI:

FROM ORDER

INNER JOIN CUSTOMER

ON ORDER.customerId=

CUSTOMER.customerId

Your configuration uses:

ansi-no-parenthesis

so OFBiz creates cleaner SQL.

Chunk 18: Table Type

Code:

table-type="InnoDB"

When OFBiz creates tables:

```
CREATE TABLE PRODUCT(
```

```
...
```

```
)
```

```
ENGINE=InnoDB
```

Why InnoDB?

Because InnoDB supports:

- transactions
- rollback
- foreign keys
- row locking

Without InnoDB:

Insert failed midway

Rollback may not work.

Chunk 19: Character Encoding

Code:

```
character-set="utf8mb4"
```

```
collate="utf8mb4_0900_ai_ci"
```

This controls text storage.

utf8mb4 supports:

English

Hindi

Chinese

Emoji 🤔

Internally MySQL stores:

Mahadev 🙏

as Unicode bytes.

Without utf8mb4:

Mahadev ?

or corrupted characters.

`collate`

Controls sorting/comparison.

Example:

Database contains:

Apple

apple

APPLE

`ai_ci` means:

- ai = accent insensitive
- ci = case insensitive

So:

```
SELECT *
```

```
FROM PRODUCT
```

```
WHERE name='apple'
```

returns:

Apple

apple

Chunk 20: Big Internal Flow (Most Important)

Now let us trace one real request.

Suppose code:

```
delegator.findOne(  
  "Product",  
  UtilMisc.toMap(  
    "productId",  
    "P100"  
  ))
```

Internal execution:

Step 1:

Delegator receives request

↓

Step 2:

Delegator checks model reader

Find entity Product



Step 3:

Group reader checks group

Product



org.apache.ofbiz



Step 4:

Delegator checks datasource

org.apache.ofbiz



localmysql



Step 5:

Datasource calls helper

GenericHelperDAO



Step 6:

Helper creates SQL

```
SELECT *  
FROM PRODUCT  
WHERE PRODUCT_ID='P100'
```



Step 7:

Connection pool provides connection

Connection conn



Step 8:

PreparedStatement created



Step 9:

Database executes query



Step 10:

ResultSet returned



Step 11:

Convert ResultSet → GenericValue



Step 12:

Return to application

That entire chain happens in milliseconds.

Chunk 21: inline-jdbc

Code:

```
<inline-jdbc
```

```
    jdbc-driver="com.mysql.cj.jdbc.Driver"
```

```
    jdbc-uri="jdbc:mysql://127.0.0.1/hotwax_commerce?autoReconnect=true&useSSL=false&characterEncoding=UTF-8&scrollTolerantForwardOnly=true&allowPublicKeyRetrieval=true"
```

```
    jdbc-username="root"
```

```
    jdbc-password="Mahadev@2402"
```

```
    isolation-level="ReadCommitted"
```

```
    pool-minsize="2"
```

```
    pool-maxsize="250"
```

```
    time-between-eviction-runs-millis="600000"/>
```

This is where actual database connection information lives.

Think:

Delegator



Datasource



inline-jdbc



MySQL Database

Without this section:

delegator.findOne()

would not know where the database exists.

Chunk 22: JDBC Driver Internal Working

Code:

```
jdbc-driver="com.mysql.cj.jdbc.Driver"
```

This tells Java:

```
"Use MySQL's JDBC implementation."
```

Internally:

OFBiz loads the class dynamically:

```
Class.forName(
```

```
"com.mysql.cj.jdbc.Driver"
```

```
);
```

When loaded:

MySQL driver automatically registers itself:

```
DriverManager.registerDriver(
```

```
new Driver()
```

```
);
```

Now Java knows:

Whenever MySQL URL appears,

use this driver.

Internal flow:

Load driver class



Driver object created



Register with DriverManager



Ready to create connections

Chunk 23: JDBC URI Internal Working

Code:

`jdbc:mysql://127.0.0.1/hotwax_commerce`

Breakdown:

`jdbc:`

Means:

Use JDBC protocol

`mysql:`

Means:

Use MySQL driver

`127.0.0.1`

Means:

Database server location

127.0.0.1:

Current machine

hotwax_commerce

Database name.

Internally:

Driver creates:

Socket connection

like:

Application

↓

TCP Socket

↓

MySQL Server

Usually on:

Port 3306

Chunk 24: Connection Pool Internal Working

Code:

```
pool-minsize="2"
```

```
pool-maxsize="250"
```

This is extremely important.

Without connection pool:

For every request:

```
Connection conn=
```

```
DriverManager.getConnection();
```

Database work:

```
conn.close();
```

Again:

```
DriverManager.getConnection();
```

Problem:

Creating connections is expensive.

Because internally:

Authenticate user

Create TCP socket

Allocate memory

Initialize session

This takes time.

Instead OFBiz creates pool:

Startup:

Pool

Connection1

Connection2

because:

pool-minsize="2"

Suppose application receives:

delegator.findOne()

Internally:

connection=

pool.borrowConnection()

Connection removed:

Pool

Connection2

Use connection:

executeQuery()

Return:

pool.returnConnection()

Pool:

Connection1

Connection2

No new connection creation.

Huge performance improvement.

Chunk 25: What happens if all connections are busy?

Suppose:

`pool-maxsize="250"`

Pool already has:

250 active connections

New request arrives:

`delegator.findList()`

Internally:

Pool checks:

`if(activeConnections<250)`

If true:

`createConnection()`

Else:

`wait()`

Request waits until someone returns connection.

Flow:

Request



Pool has free connection?

Yes → Give connection

No



Can create more?

Yes → Create connection

No



Wait

Chunk 26: Eviction Process

Code:

```
time-between-eviction-runs-millis="600000"
```

Value:

600000 ms

=

10 minutes

Internally background thread runs:

EvictionThread

every:

10 minutes

Checks:

for(Connection c:pool)

If:

c.idleTime > threshold

then:

close()

remove()

Why?

Otherwise:

Unused connections

remain forever.

Memory waste occurs.

Chunk 27: Transaction Isolation Level

Code:

```
isolation-level="ReadCommitted"
```

This controls how transactions see other transaction data.

Suppose:

Transaction A:

Update balance=5000

but not committed.

At same time:

Transaction B:

Read balance

Question:

Should B see:

5000

or old value?

With:

ReadCommitted

B only sees committed data.

Internally:

Java executes:

```
connection.setTransactionIsolation(  
    Connection.TRANSACTION_READ_COMMITTED  
);
```

Example:

Transaction A:

Balance=1000

Update:

Balance=5000

Not committed yet.

Transaction B:

SELECT balance

still sees:

1000

After commit:

5000

becomes visible.

Prevents:

Dirty Reads

Chunk 28: Complete Transaction Lifecycle

Suppose service:

`createOrder()`

contains:

`insertOrder()`

`updateInventory()`

`createPayment()`

Internally:

OFBiz transaction manager does:

Step 1

Start transaction

`tx.begin();`

Database:

Transaction ID=101

created.

Step 2

Disable auto commit

`connection.setAutoCommit(false);`

Normally:

INSERT

immediately saves.

Now it waits.

Step 3

Execute operations

insertOrder()

↓

updateInventory()

↓

createPayment()

Step 4A Success path

tx.commit();

Database permanently saves everything.

Flow:

Begin

↓

Insert order

↓

Update inventory

↓

Create payment

↓

Commit

Step 4B Failure path

Suppose:

`createPayment()`

fails.

Transaction manager:

`tx.rollback();`

Database restores previous state:

Remove order

Restore inventory

Undo changes

Flow:

Begin

↓

Insert order

↓

Update inventory

↓

Payment fails

↓

Rollback

↓

Undo all operations

Final Big Picture (Entire OFBiz Flow)

Application Code

`delegator.findOne()`



Delegator



Entity Model Reader



Entity Group Reader



Datasource



GenericHelperDAO



Connection Pool



JDBC Driver



MySQL Server



ResultSet



GenericValue



Application

This entire chain is what happens when even a simple line like:

```
delegator.findOne("Product")
```

executes.